Software Requirements Specification (SRS)

Nothaltesystem für einen autonom fahrenden Miniatur-LKW

Projektinformationen

Modul:	Software Engineering (Program Design)
Hochschule:	DHBW Ravensburg, Campus Friedrichshafen
Studiengang:	Embedded Systems
Betreuer:	Daniel Stimac

Vier Muske(l)tiere

Dominik Borowski	(7373896)
Niklas Grunewald	(5474255)
Carla Klammt	(1055614)
Elias Sehmsdorf	(2369656)

Friedrichshafen, 08. Dezember 2025

Contents

1 Einleitung **1**
1.1 Zweck des Dokuments 1
1.2 Geltungsbereich des Systems 1

2 Systemübersicht **2**
2.1 Produktperspektive 2
2.2 Systemkontext 2
2.2.1 Akteur 3
2.2.2 Externe Systemkomponenten 3
2.3 Schnittstellenübersicht 3

3 Funktionale Anforderungen (FR-xx) **4**
3.1 Hardstop (Pilztaster) 4
3.2 Softstop (seitlicher Taster + Software) 5
3.3 Emergency Stop (E-Stop) – softwarebasierte Hinderniserkennung 6
3.4 Warnmodus / Fehlerszenarien 7
3.5 Betriebsarten: automatische vs. manuelle Freigabe 8
3.6 Integration in die bestehende ROS-/Arduino-Architektur 8
3.7 Logging 9
3.8 Warnleuchten 9

4 Nicht-funktionale Anforderungen **10**
4.1 Architektur 10
4.2 Zeitverhalten & Performance 10
4.3 Sicherheit 11
4.4 Zuverlässigkeit 11
4.5 Logging 11

5 Use-Case-Modell **12**
5.1 Use-Case-Diagramm 12
5.2 Beschreibung 12
5.3 Akteure 12
5.4 Beschreibung der Use Cases 13

6 Unified Modeling Language (UML)-Verhaltesnmodelle **15**
6.1 Zustandsdiagramm "Nothaltesystem" 15
6.1.1 Interner Zustandsautomat "Softstop" 19
6.1.2 Interner Zustandsautomat "E-Stop" 22
6.2 Aktivitätsdiagramm „Nothalt auslösen“ 24
6.3 Sequenzdiagramme 28
6.3.1 Sensorfehler/Warnmodus 28
6.3.2 Softstop-Sequenz 31
6.3.3 E-Stop-Sequenz 34
6.3.4 Quittierung & Kriechmodus 37

7 Systemarchitektur **40**
7.1 Komponentenübersicht 40
7.2 Daten- & Funktionsflüsse 41

8 Qualitätssicherung der Anforderungen **42**
8.1 Kriterien: Vollständigkeit, Eindeutigkeit, Konsistenz 42
8.2 Vorgehen zur Qualitätssicherung 43
8.3 Verifikation der Anforderungen 44
8.4 Nachvollziehbarkeit (Traceability) 45

Definitionen **46**

1 Einleitung

1.1 Zweck des Dokuments

Dieses Dokument spezifiziert vollständig, eindeutig und überprüfbar das Nothaltesystem eines autonom fahrenden Miniatur-LKWs, das im Rahmen der Projektarbeit im Modul Software Engineering an der DHBW Friedrichshafen entwickelt wurde. Ziel des Dokuments ist es, alle funktionalen und nicht-funktionalen Anforderungen zu definieren, das Systemverhalten in unterschiedlichen Betriebszuständen klar darzustellen und die Interaktion zwischen Softwarekomponenten, Sensorsystemen und Aktorik präzise zu beschreiben. Die Spezifikation dient als Grundlage für Implementierung, Test, Validierung sowie sicherheitsrelevante Bewertung des Systems.

1.2 Geltungsbereich des Systems

Das in diesem Dokument beschriebene Nothaltesystem umfasst ausschließlich sicherheitsrelevante Funktionen zur Erkennung gefährlicher Situationen und zur unmittelbaren Einleitung geeigneter Reaktionen. Dazu gehören:

- Physicher Not-Aus über Pilztaster; verriegelt das System und setzt Motorcontroller sofort außer Betrieb (Hardstop)
- Kontrollierter Stopp des Fahrzeugs durch Taster oder User Interface (UI); keine Systemverriegelung (Softstop)
- Automatischer Notstopp durch Sensorik bei kritischem Hindernisabstand (E-Stop)
- Quittierung und Wiederanlauf (Wiederaufnahme des Fahrbetriebs durch automatische oder manuelle Freigabe (Resume))
- Langsame, begrenzte Bewegung nach einem Nothalt zur sicheren Situationsklärung. (Kriechmodus)
- Warnsignale (Light Emitting Diode (LED), Dachleuchte)
- Protokollierung sicherheitsrelevanter Ereignisse

Nicht Bestandteil des Systems sind:

- allgemeine Fahrzeugsteuerung

- Navigationsalgorithmen
- Hindernisvermeidung
- Odometrie
- Fahrdynamikregelung

Das Nothaltesystem arbeitet ergänzend zum autonomen Fahrsystem, besitzt jedoch Vorrang bei sicherheitskritischen Situationen.

2 Systemübersicht

2.1 Produktperspektive

Das Nothaltesystem ist ein sicherheitskritisches Subsystem des autonomen Miniatur-LKWs. Es arbeitet parallel zur Fahrzeugsteuerung und überwacht kontinuierlich Sensorwerte sowie Operatorbefehle. Bei Erreichen definierter Gefährdungszustände greift es mit höchster Priorität ein und überschreibt alle anderen Fahrfunktionen. Das System interagiert über ROS, CAN-Kommunikation und Aktor-Schnittstellen mit bestehenden Komponenten wie Lokalisierung, Fahrsteuerung, LiDAR-Sensorik und Motorregelung. Es ersetzt nicht die autonome Fahrlogik, sondern ergänzt diese um eine deterministische Notfall- und Sicherheitsmechanik.

2.2 Systemkontext

Der Systemkontext beschreibt alle externen Komponenten, Akteure und Subsysteme, die mit dem Nothaltesystem interagieren.

2.2.1 Akteur

Table 1: Menschlicher Akteur des Nothaltesystems

Akteur	Beschreibung
Fahrer/Operator	Der Operator kann über verschiedene Eingabegeräte Steuerbefehle, Softstop-Signale und Fahrkommandos an das Nothaltesystem bzw. die Fahrzeugsteuerung senden. Er ist der primäre Mensch–Maschine-Interaktionspartner.

2.2.2 Externe Systemkomponenten

Table 2: Externe Systemkomponenten im Kontext des Nothaltesystems

Komponente	Rolle / Interaktion
Fernbedienung	Sendet Fahrbefehle und Softstop-Signale an das Nothaltesystem oder das autonome Fahrsystem.
Autonomes Fahrsystem	Übermittelt Fahrbefehle und erhält den Nothaltstatus zurück.
`ArduinoMaster`	Übergibt den softwareseitigen Nothaltstatus an das hardwarenahe System.
`ArduinoSlave`	Setzt Stop-Signale in PWM-Signale um und steuert die Motoren entsprechend.
Antrieb / Motorcontroller	Führt Brems- bzw. Stoppbefehle physikalisch aus.
UI / RViz	Visualisiert Nothaltstatus, Warnungen und ermöglicht ggf. Softstop-Befehle.
Logging / ROS-Bags	Zeichnet Nothalt-Ereignisse und Statusmeldungen für Analysezwecke auf.
Dachwarnleuchte	Erhält ein An-/Aus-Signal zur optischen Warnung bei aktivem Nothalt.
Sensorik (LiDAR, Encoder)	Liefert Umgebungsdaten zur autonomen Erkennung von Gefahrensituationen.

2.3 Schnittstellenübersicht

Der Systemkontext weist mehrere klar abgegrenzte Schnittstellen zwischen Nothaltesystem, Fahrzeugsteuerung, Sensorik und Peripherie auf. Die wichtigsten logischen Schnittstellen sind in Tabelle 3 zusammengefasst.

Table 3: Hauptschnittstellen des Nothaltesystems

Schnittstelle	Beschreibung
Fahrbefehle	Von Fernbedienung oder autonomem Fahrsystem in Richtung Nothaltesystem. Die Fahrbefehle umfassen u. a. Geschwindigkeits-, Richtungs- und Betriebsmodusvorgaben.
Nothaltstatus (ROS → Arduino)	Übergibt sicherheitskritische Nothaltinformationen vom ROS-basierten Nothaltesystem an das Arduino-System, das anschließend den physischen Stopp auslöst (z. B. Abschalten der Motorfreigabe).
CAN-Signale	Weiterleitung von E-Stop-Informationen zu den relevanten Fahrzeugsteuerungen über den CAN-Bus, damit auch andere Steuergeräte den Nothaltzustand berücksichtigen können.
Warnsignal (Dachleuchte)	Einfaches Ausgangssignal zur optischen Kennzeichnung eines aktiven Nothalts. Bei gesetztem Nothaltstatus wird die Dachwarnleuchte dauerhaft aktiviert.
Sensorströme (Lidar, Encoder)	Eingehende Sensordatenströme, die zur automatisierten Hinderniserkennung sowie zur Bewertung der Fahrzeugbewegung verwendet werden. Dazu zählen insbesondere 2D-/3D-Lidar-Scans und Encoder-Rückmeldungen.
Logging-Daten	Ereignisdaten und Sensordaten, die zur späteren Analyse, Nachvollziehbarkeit und Validierung des Sicherheitsverhaltens persistiert werden (z. B. als ROS-Bag-Dateien).

Durch diese Abgrenzung wird deutlich, dass das Nothaltesystem als zentrale sicherheitskritische Instanz zwischen Bediener, Fahrzeugregelung, Sensorik und sicherheitsrelevanten Aktoren fungiert.

3 Funktionale Anforderungen (FR-xx)

3.1 Hardstop (Pilztaster)

FR-HS-01

Das System muss eine Hardstop-Funktion bereitstellen, die durch den Pilztaster ausgelöst wird.

FR-HS-02

Beim Hardstop muss der Slave-Arduino den Motor-Pulse Width Modulation; Pulsweiten-

modulation zur Steuerung von Motoren und Aktoren (PWM)-Wert sofort auf den kalibrierten Stop-Wert setzen, um die maximale Verzögerung sicherzustellen.

FR-HS-03
Der Hardstop muss höchste Priorität besitzen und alle anderen Steuerbefehle außer Kraft setzen (manuell, autonom, Softstop, E-Stop).

FR-HS-04
Das Fahrzeug darf eine Weiterfahrt erst dann erlauben, wenn:

- der Pilztaster mechanisch zurückgestellt wurde und
- der Hardstop-Zustand softwareseitig zurückgesetzt wurde.

FR-HS-05
Während eines Hardstop müssen die Dachwarnleuchten dauerhaft aktiviert sein.

3.2 Softstop (seitlicher Taster + Software)

FR-SS-01
Das System muss einen Softstop bereitstellen, der durch den seitlichen Nothalt-Taster oder ein Softwarekommando ausgelöst werden kann.

FR-SS-02
Beim Softstop muss das Fahrzeug kontrolliert bis zum Stillstand abgebremst werden.

FR-SS-03
Im Softstop- und Emergency-Stop-Zustand muss das Fahrzeug folgende eingeschränkte Bewegungen erlauben:

1. Rückwärtsfahrt und Lenken jederzeit erlaubt.
2. Vorwärtsfahrt nur im Kriechmodus, wenn
    - der gemessene Abstand zum Hindernis größer als 10 cm ist und
    - der Bediener aktiv eine geringe Vorwärtsbewegung anfordert.
3. Wenn der Abstand $\leq$ 10 cm beträgt, muss Vorwärtsfahrt vollständig blockiert werden.

FR-SS-04
Der seitliche Taster muss den Softstop-Zustand anzeigen:

- AUS: kein Softstop aktiv,
- ROT (dauerhaft): Softstop aktiv, Bewegungsrestriktionen aktiv,
- ROT (blinkend, Frequenz: 2 Hz): eingeschränkter Bewegungmodus nach Benutzerbestätigung.

FR-SS-05
Ein Hardstop muss einen Softstop jederzeit übersteuern können.

FR-SS-06
Alle Funktionen des Softstops (Auslösen, Aufheben, Umschalten in Kriechmodus) müssen sowohl

- über den seitlichen Nothalt-Taster am Fahrzeug als auch
- über die Benutzerschnittstelle (UI/RViz)

auslösbar sein.

3.3 Emergency Stop (E-Stop) – softwarebasierte Hinderniserkennung

FR-ES-01
Das System muss einen automatischen Emergency Stop auslösen, wenn die Hinderniserkennung ein Objekt innerhalb der kritischen Zone erkennt.

FR-ES-02
Ein Emergency Stop muss in denselben Zustand führen wie ein Softstop (Bremsung, Bewegungseinschränkung, Tasteranzeige).

FR-ES-03
Für die Hinderniserkennung müssen sowohl der 2D-Laserscanner (LaserScan) als auch der 3D-Lidar (PointCloud2) berücksichtigt werden.

FR-ES-04
In der kritischen Zone muss eine ODER-Logik gelten: Ein Emergency Stop ist auszulösen,

sobald mindestens einer der beiden Sensoren ein Hindernis innerhalb der kritischen Entfernung meldet.

FR-ES-05
Es müssen mindestens zwei Abstandszonen definiert werden:

- Kritische Zone: Emergency Stop wird ausgelöst.
- Warnzone: Es wird eine Vorwarnung ausgegeben, jedoch noch kein Bremsvorgang eingeleitet.

FR-ES-06
Der 3D-Lidar muss softwareseitig auf einen relevanten Sichtbereich (Field-of-View) beschränkt werden, um Fehlalarme von Objekten oberhalb oder außerhalb des Fahrweges zu reduzieren.

FR-ES-07
Wenn zwischen Hinderniserkennung und notwendigem Bremsbeginn mindestens ein Zyklus der Warnsignale aktualisiert werden kann, muss das System vor Beginn des Bremsvorgangs ein Warnsignal ausgeben.

3.4 Warnmodus / Fehlerszenarien

FR-WM-01
Das System muss den Zustand aller für den Emergency Stop verwendeten Sensoren kontinuierlich überwachen.

FR-WM-02
Bei Ausfall oder ungültigen Messwerten eines Sensors muss das System in den Warnmodus wechseln.

FR-WM-03
Im Warnmodus muss das System:

- fehlerhafte Sensoren ignorieren,
- verbleibende Sensoren verwenden,
- den Bediener sichtbar informieren (z. B. UI, RViz oder Statusanzeige).

FR-WM-04
Fallen alle Sensoren für den Emergency Stop aus, muss das System autonome Fahrmodi blockieren und den Bediener auffordern, die Einschränkung zu bestätigen.

3.5 Betriebsarten: automatische vs. manuelle Freigabe

FR-RM-01
Das System muss zwei Freigabemodi unterstützen: automatische Freigabe und manuelle Freigabe.

FR-RM-02
Im Modus „automatische Freigabe“ muss ein Softstop/E-Stop aufgehoben werden, sobald

- das Fahrzeug vollständig steht und
- kein Hindernis mehr in der Nothaltzone erkannt wird.

FR-RM-03
Im Modus „manuelle Freigabe“ darf ein Softstop/E-Stop nur durch eine Benutzeraktion aufgehoben werden (seitlicher Taster oder UI-Kommando).

FR-RM-04
Ein Hardstop darf niemals automatisch aufgehoben werden.

3.6 Integration in die bestehende ROS-/Arduino-Architektur

FR-INT-01
Die Nothalt-Logik muss als ROS-Node auf dem Hauptrechner (Linux, Jetson/NUC) implementiert werden.

FR-INT-02
Der Nothaltzustand muss über ein ROS-Topic veröffentlicht werden, das sowohl vom Truck Bridge Node als auch direkt von ROS-fähigen Arduinos verarbeitet werden kann.

FR-INT-03
Statt UDP-Kommunikation sollen die Arduinos den Nothaltzustand über ROS-Nachrichten empfangen.

FR-INT-04
Die interne Kommunikation zwischen Master- und Slave-Arduino kann weiterhin über den CAN-Bus erfolgen, muss jedoch durch ROS-basierte Zustandsnachrichten ergänzt werden.

FR-INT-05
Der Slave-Arduino muss die empfangenen ROS-Zustandsnachrichten direkt in Motor-PWM-Befehle umsetzen.

3.7 Logging

FR-LOG-01
Das System muss alle Nothalt-Ereignisse (Softstop, Hardstop, Emergency Stop) protokollieren.

FR-LOG-02
Jedes Nothalt-Ereignis muss mindestens enthalten:

- Zeitstempel,
- Auslösequelle (Pilz, Taster, Software, Sensorik),
- Sensordaten zum Auslösezeitpunkt (soweit verfügbar),
- Fahrzeugpose und Steuerungsmodus.

FR-LOG-03
Die Protokolle müssen für Entwickler eindeutig nachvollziehbar sein, um Fehlersituationen analysieren zu können.

3.8 Warnleuchten

FR-WL-01
Die Dachwarnleuchten müssen bei jedem aktiven Nothaltzustand (Hardstop, Softstop, Emergency Stop) dauerhaft eingeschaltet sein.

FR-WL-02
Betritt ein Hindernis die definierte Warnzone, müssen die Dachwarnleuchten als Frühwarnsignal eingeschaltet werden, noch bevor eine Bremsung eingeleitet wird.

FR-WL-03
Die Warnleuchten dürfen erst dann deaktiviert werden, wenn

- kein Nothaltzustand mehr aktiv ist und
- keine Warnzone-Bedingung mehr erfüllt ist.

4 Nicht-funktionale Anforderungen

4.1 Architektur

NFR-ARC-01
Die Arduinos müssen ROS-kompatible Software ausführen (z. B. micro-ROS), sodass sie vollständig in die bestehende ROS1/ROS2-basierten Systemarchitektur integriert sind.

NFR-ARC-02
Alle Nothalt-relevanten Meldungen müssen als ROS-Nachrichten veröffentlicht werden (z. B. `/emergency_stop_state`, `/fault_state`, `/sensor_health`).

4.2 Zeitverhalten & Performance

NFR-TIM-01
Zwischen der Erkennung eines Hindernisses in der kritischen Zone und dem Versand des Nothaltkommandos an den Master-Arduino dürfen maximal 50 ms vergehen.

NFR-TIM-02
Das System muss sicherstellen, dass das Fahrzeug bei einer Geschwindigkeit von bis zu 0,55 m/s vor einem Hindernis zum Stillstand kommt und dabei einen Sicherheitsabstand von mindestens 20 cm einhält.

NFR-TIM-03
Die Sensorverarbeitung muss mindestens folgende Raten unterstützen:

- 2D-Laserscanner: 40 Hz,
- 3D-Lidar: 20–30 Hz,
- Encoder: 50 Hz.

4.3 Sicherheit

NFR-SAF-01
Im Falle widersprüchlicher oder fehlender Sensordaten muss das System sicher reagieren, indem es entweder in den Warnmodus wechselt oder einen Nothalt durchführt (je nach Situation).

NFR-SAF-02
Die Hardstop-Funktion muss unabhängig von der Software zuverlässig funktionieren.

4.4 Zuverlässigkeit

NFR-REL-01
Das System soll so ausgelegt sein, dass Fehlbremsungen (False Positives) höchstens 5 % betragen.

NFR-REL-02
Nicht erkannte relevante Hindernisse (False Negatives) dürfen höchstens 1 % betragen.

4.5 Logging

NFR-LOG-01
Das Logging-System muss ROS-Bag-Dateien als primäres Speicherformat verwenden.

NFR-LOG-02
Logdaten müssen so lange gespeichert werden, bis

- der Benutzer sie manuell löscht oder
- sie durch eine automatische Aufräumroutine gelöscht werden.

Die automatische Aufräumroutine muss Logdateien löschen, die älter als 14 Tage sind.

NFR-LOG-03
Arduinos müssen Sensordaten und Zustandsdaten über ROS-Topics bereitstellen, sodass sie automatisch in ROS-Bags gespeichert werden können.

5 Use-Case-Modell

5.1 Use-Case-Diagramm

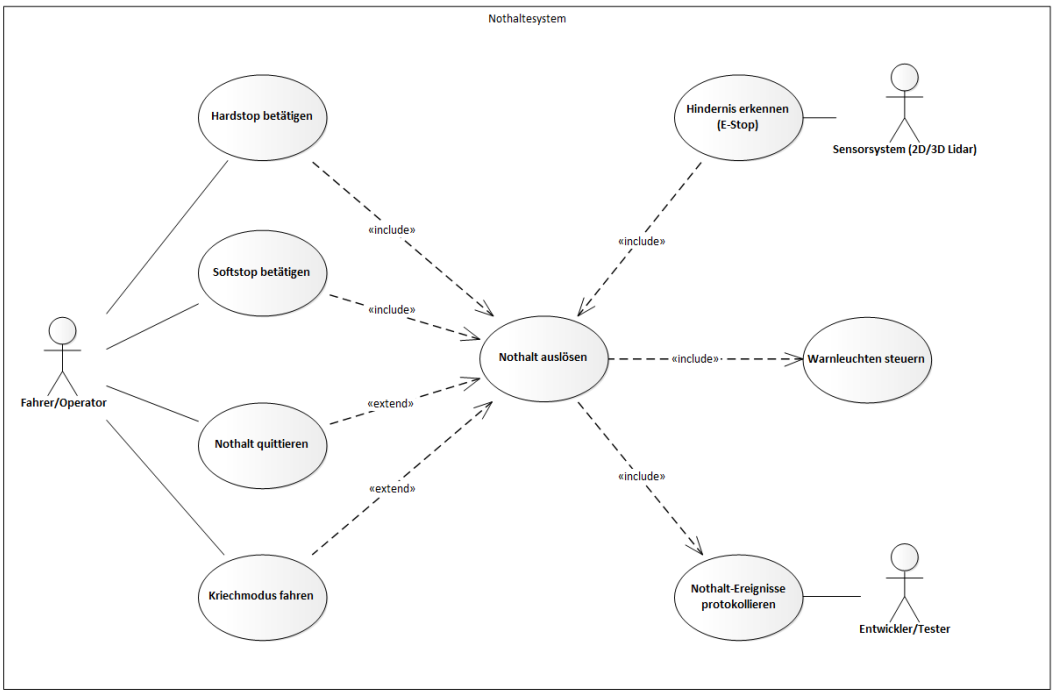

Figure 1: Use-Case-Diagramm des Nothaltesystems

5.2 Beschreibung

Das Use-Case-Diagramm zeigt alle relevanten Interaktionen, beteiligten Akteure sowie die funktionalen Abläufe, die im Rahmen des Nothaltesystems eines autonomen Nutzfahrzeugs unterstützt werden. Ziel des Modells ist es, die externen Sichtweisen auf das System klar zu strukturieren und die wesentlichen Funktionen – insbesondere das Auslösen und Verarbeiten eines Nothalts – übersichtlich darzustellen.

5.3 Akteure

Fahrer / Operator

Der Fahrer bzw. Operator ist der primäre menschliche Nutzer des Systems. Er kann

Hardstop und Softstop auslösen, den Nothalt quittieren sowie den Kriechmodus für kontrollierte Bewegungen nach einem Nothalt anfordern.

Sensorsystem (2D/3D LiDAR)
Das Sensorsystem repräsentiert die automatisierte Hinderniserkennung. Es agiert als externer Auslöser eines Nothalts, indem es kritische Objekte oder Gefahren erkennt und dadurch die Funktion „Nothalt auslösen“ anstößt.

Entwickler / Tester
Dieser Akteur steht für Personal, das das System überwacht, Tests durchführt und Zugang zu den protokollierten Nothalt-Ereignissen benötigt.

5.4 Beschreibung der Use Cases

Hardstop betätigen
Der Hardstop wird durch Betätigung eines Not-Aus-Tasters aktiviert. Diese Aktion führt zwingend zur Aktivität „Nothalt auslösen“. Der Hardstop ist ein sicherheitskritisches, sofort wirkendes Ereignis.
Beziehung: include → Nothalt auslösen

Softstop betätigen
Der Softstop wird über ein Bedienelement ausgelöst und führt ebenfalls zu einem Nothalt, jedoch ohne sofortiges Trennen der Energieversorgung.
Beziehung: include → Nothalt auslösen

Hindernis erkennen (E-Stop)
Das Sensorsystem erkennt ein Hindernis, erzeugt ein E-Stop-Signal und initiiert unmittelbar die Funktion „Nothalt auslösen“.
Beziehung: include → Nothalt auslösen

Nothalt auslösen
Dies ist der zentrale Use Case des Systems. Er umfasst alle sicherheitsrelevanten Reaktionen wie das sofortige Stoppen des Fahrzeugs, das Aktivieren der Warnleuchten sowie das Protokollieren relevanter Ereignisdaten.
Beziehungen:

- include → Warnleuchten steuern

- include → Nothalt-Ereignisse protokollieren

Warnleuchten steuern
Dieser Use Case aktiviert oder deaktiviert externe optische Signale, welche den Nothalt nach außen anzeigen.

Nothalt-Ereignisse protokollieren
Hierbei werden Zeitpunkt, Auslöser, Systemstatus und weitere sicherheitsrelevante Informationen aufgezeichnet. Entwickler und Tester nutzen diese Daten für Diagnose und Validierung.

Nothalt quittieren
Der Operator bestätigt nach einem Nothalt, dass er die Situation bewertet hat. Die Quittierung erweitert den zentralen Use Case „Nothalt auslösen“, da sie nur nach einem ausgelösten Nothalt auftreten kann.
Beziehung: extend ← Nothalt auslösen

Kriechmodus fahren
Nach erfolgter Quittierung kann der Operator ein eingeschränktes und kontrolliertes Fahren im Kriechmodus anfordern. Dieser Use Case erweitert ausschließlich Situationen nach einem ausgelösten Nothalt.
Beziehung: extend ← Nothalt auslösen

6 UML-Verhaltesnmodelle

6.1 Zustandsdiagramm "Nothaltesystem"

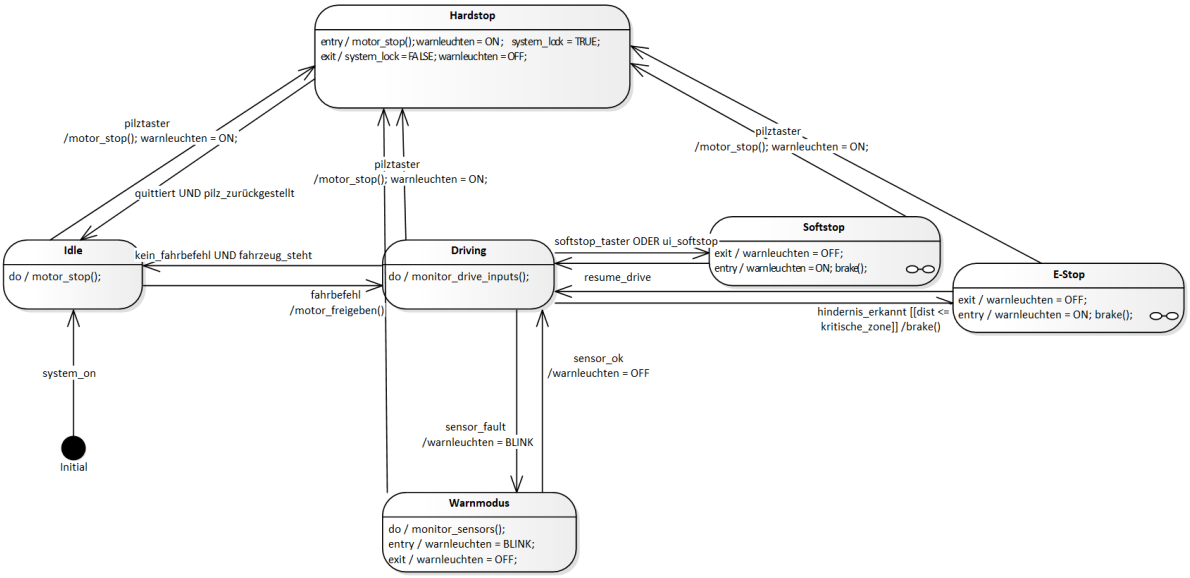

Figure 2: Zustandsdiagramm des Nothaltesystems

Das Zustandsdiagramm beschreibt das dynamische Verhalten des Nothaltesystems und zeigt, wie es auf interne und externe Ereignisse reagiert. Im Mittelpunkt stehen die sicherheitskritischen Zustände Hardstop, Softstop, E-Stop, Idle, Driving und Warnmodus. Jeder Zustand besitzt spezifische Entry- und Exit-Aktionen mit direkter Auswirkung auf Motorsteuerung, Warnsignale und Systemlogik.

Ziel der Modellierung ist es, das deterministische Verhalten des Systems transparent und prüfbar zu machen, insbesondere die Priorisierung von Hardstop und E-Stop gegenüber normalen Fahrzuständen.

Initialer Zustand Nach dem Einschalten des Systems (*system_on*) wird automatisch der Zustand **Idle** eingenommen. Dies entspricht einem sicheren Grundzustand, in dem die Motorsteuerung deaktiviert bleibt.

Zustand: Idle Aktionen:

- do / motor_stop();

Der Idle-Zustand repräsentiert einen sicheren Ruhezustand. Das Fahrzeug ist nicht freigegeben und kann keine Bewegungsbefehle ausführen.

Übergänge:

- system_on → Idle
- fahrbefehl **UND** fahrzeug_steht → Driving
- pilztaster → Hardstop
- sensor_fault → Warnmodus
- quittiert **UND** pilz_zurückgestellt → Idle

Zustand: Driving Aktionen:

- do / monitor_drive_inputs();

Im Driving-Zustand werden Fahrzeugbewegungen erlaubt und die Fahrbefehle kontinuierlich überwacht.

Übergänge:

- pilztaster → Hardstop
- softstop_taster **ODER** ui_softstop → Softstop
- hinderis_erkannt **UND** dist $\leq$ kritische_zone → E-Stop
- kein_fahrbefehl **UND** fahrzeug_steht → Idle
- sensor_fault → Warnmodus

Zustand: Hardstop Entry:

- motor_stop()
- warnleuchten = ON
- system_lock = TRUE

Exit:

- system_lock = FALSE
- warnleuchten = OFF

Der Hardstop ist der sicherste und restriktivste Zustand. Er kann nur manuell ausgelöst werden und verriegelt das System, bis der Pilztaster zurückgestellt und der Nothalt quittiert wurde.

Übergänge:

- quittiert **UND** pilz_zurückgestellt → Idle
- Von jedem Zustand erreichbar über: pilztaster

Zustand: Softstop Entry:

- warnleuchten = ON
- brake()

Exit:

- warnleuchten = OFF

Der Softstop ist ein kontrollierter Stopp ohne Systemverriegelung. Er kann über seitlichen Taster oder UI ausgelöst werden.

Übergang zurück:

- resume_drive → Driving

Zustand: E-Stop Entry:

- warnleuchten = ON
- brake()

Exit:

- warnleuchten = OFF

Der E-Stop wird automatisch ausgelöst, wenn ein Hindernis in der kritischen Zone erkannt wird. Er stoppt das Fahrzeug unmittelbar, verriegelt das System jedoch nicht wie ein Hardstop.

Übergänge:

- Nach Verlassen der Gefahr → Driving oder Idle

Zustand: Warnmodus Aktionen:

- do / monitor_sensors();
- entry / warnleuchten = BLINK;
- exit / warnleuchten = OFF;

Der Warnmodus wird aktiviert, wenn ein Sensorfehler auftritt. Das Fahrzeug bleibt stehen und signalisiert eine Störung.

Übergänge:

- sensor_ok → Driving
- sensor_ok **UND** kein_fahrbefehl → Idle

Übergreifende Priorisierung Das Zustandsmodell definiert eine klare Priorität der sicherheitskritischen Reaktionen:

1. Hardstop (manuell, höchste Priorität)
2. Emergency Stop (automatisch, sicherheitskritisch)
3. Softstop (kontrolliert durch Operator/UI)
4. Warnmodus (bei Sensorfehlern)
5. Driving / Idle (Normalbetrieb)

Diese Priorisierung stellt sicher, dass jederzeit die sicherste Reaktion ausgeführt wird – unabhängig vom aktuellen Betriebszustand des Fahrzeugs.

6.1.1 Interner Zustandsautomat "Softstop"

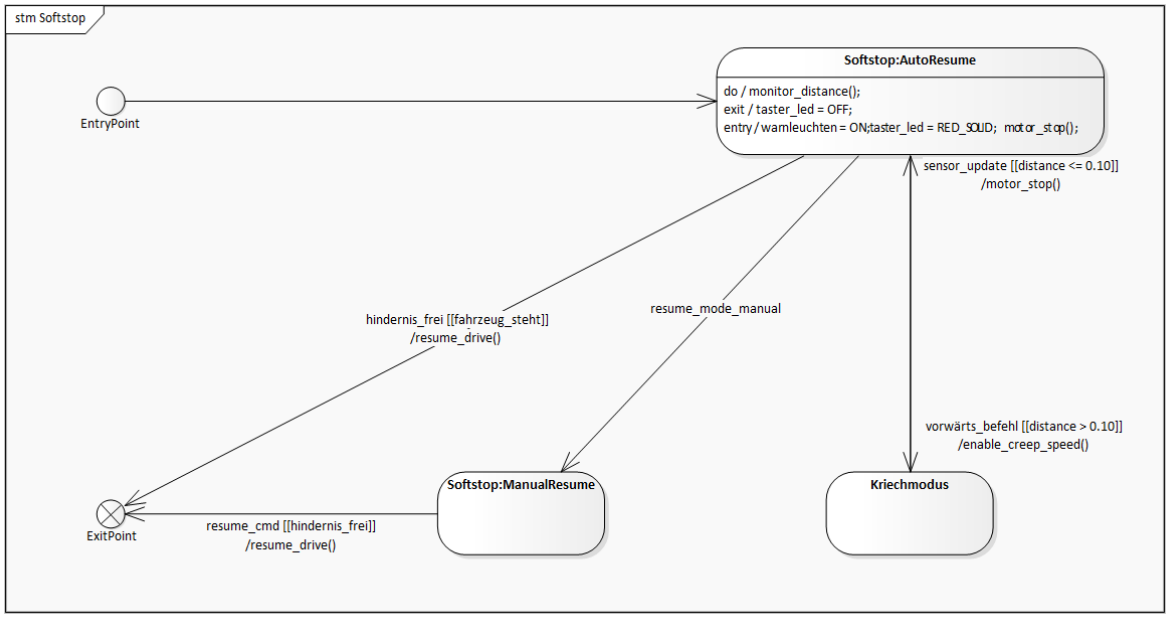

Figure 3: Zustandsdiagramm des Softstop-Moduls

Der Softstop-Zustand besitzt eine eigene, hierarchische Zustandsmaschine, die den Ablauf der Wiederaufnahme des Fahrbetriebs (AutoResume und ManualResume) sowie den Übergang in den Kriechmodus detailliert beschreibt. Diese interne Finite State Machine (FSM)

verhindert unkontrolliertes Wiederanfahren und stellt sicher, dass das Fahrzeug erst dann weiterfahren darf, wenn die Umgebungssituation sicher ist.

Startpunkt
Nach Eintritt in den Softstop-Zustand beginnt die Ausführung im Zustand *Softstop::AutoResume*.

Zustand: Softstop::AutoResume Entry-Aktionen:

- warnleuchten = ON
- taster_led = RED_SOLID
- motor_stop()

Do-Aktion:

- monitor_distance() (kontinuierliche Hindernisüberwachung)

Exit-Aktion:

- taster_led = OFF

Dieser Zustand versucht automatisch festzustellen, ob nach dem Softstop die Weiterfahrt sicher möglich ist. Die automatische Wiederaufnahme wird aktiviert, wenn keine Hindernisse mehr in unmittelbarer Nähe erkannt werden.

Übergänge:

1. sensor_update [distance <= 0.10] → Self-loop zu AutoResume
   Hindernis ist zu nah → Fahrzeug bleibt weiterhin gestoppt.
2. hindernis_frei **UND** fahrzeug_steht → ExitPoint → Driving
   Aktion:
   - resume_drive()
3. resume_mode_manual → Softstop::ManualResume
   Wenn der Operator manuell übernehmen möchte (z. B. durch UI-Eingabe).
4. vorwärts_befehl [distance > 0.10] → Kriechmodus
   Der Operator fordert eine kleine Vorwärtsbewegung an → Aktivierung des Kriechmodus.

Zustand: Softstop::ManualResume Dieser Zustand erlaubt eine manuelle Wiederaufnahme durch den Operator.

Übergänge:

- resume_cmd [hindernis_frei] → ExitPoint → Driving
  Aktion:
  - resume_drive()

Der Operator bestätigt also aktiv, dass die Umgebung frei ist.

Zustand: Kriechmodus Dieser Zustand ermöglicht eine kontrollierte, stark reduzierte Bewegung des Fahrzeugs, z. B. wenn der Operator millimetergenau an ein Objekt heranfahren möchte.

Eintrittsaktion:

- enable_creep_speed()

Der Kriechmodus endet mit einem Fahrbefehl oder dem Verlassen des Softstop-Hauptzustandes.

6.1.2 Interner Zustandsautomat "E-Stop"

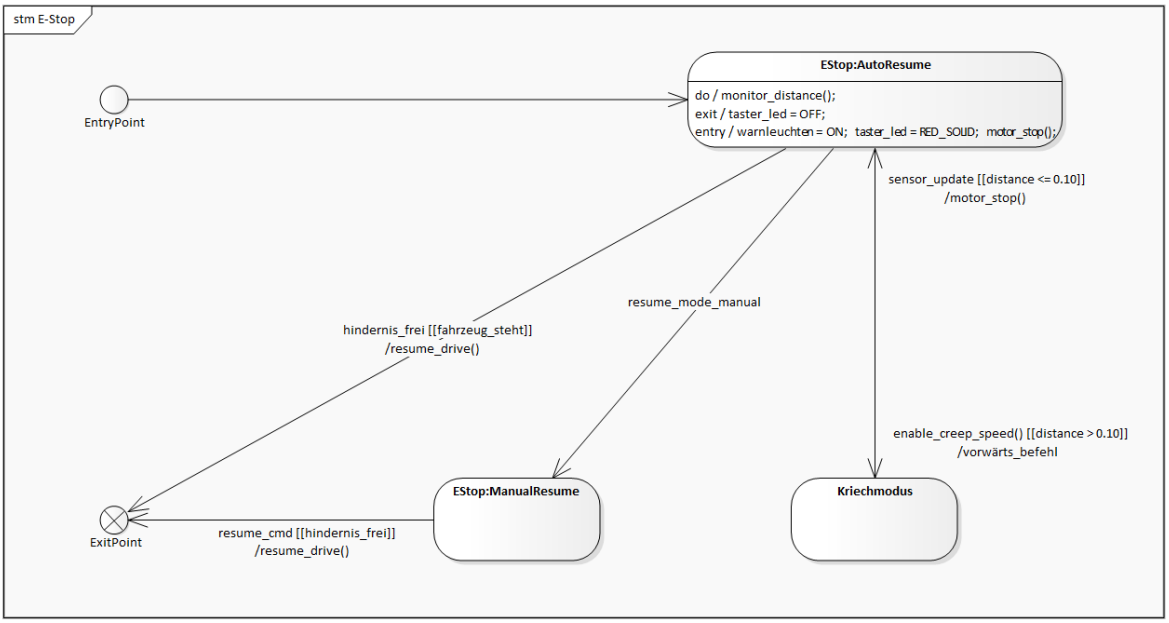

Figure 4: Zustandsdiagramm des E-Stop-Moduls

Das interne Verhalten des E-Stop-Zustands entspricht strukturell dem Softstop-Automaten, ist jedoch sicherheitskritischer. Insbesondere ist der automatische Wiederanlauf restriktiver. Das Modell zeigt, wie das System zwischen automatischer Wiederaufnahme, manueller Wiederaufnahme und Kriechmodus unterscheidet.

Startpunkt
Nach Eintritt in den E-Stop-Zustand beginnt die interne FSM im Zustand *EStop::AutoResume*.

Zustand: EStop::AutoResume Entry-Aktionen:

- warnleuchten = ON
- taster_led = RED_SOLID
- motor_stop()

Do-Aktion:

- monitor_distance()

Exit-Aktion:

- taster_led = OFF

Dieser Zustand überwacht kontinuierlich die Entfernung zum Hindernis. Im E-Stop hat Sicherheit absolute Priorität: automatische Wiederaufnahme ist nur unter klar definierten Bedingungen zulässig.

Übergänge:

1. sensor_update [distance <= 0.10] → Self-loop zu AutoResume
   Hindernis weiterhin in kritischer Nähe → Fahrzeug bleibt gestoppt.
2. hindernis_frei **UND** fahrzeug_steht → ExitPoint → Driving
   Aktion:
   - resume_drive()
3. resume_mode_manual → EStop::ManualResume
4. enable_creep_speed [distance > 0.10] → Kriechmodus
   Der Operator fordert Bewegung mit reduzierter Geschwindigkeit an.

Zustand: EStop::ManualResume Ähnlich wie beim Softstop kann der Operator aktiv entscheiden, die Fahrt wieder aufzunehmen.

Übergang:

- resume_cmd [hindernis_frei] → ExitPoint → Driving
  Aktion:
  - resume_drive()

Manuelle Bestätigung ist erforderlich

6.2 Aktivitätsdiagramm „Nothalt auslösen“

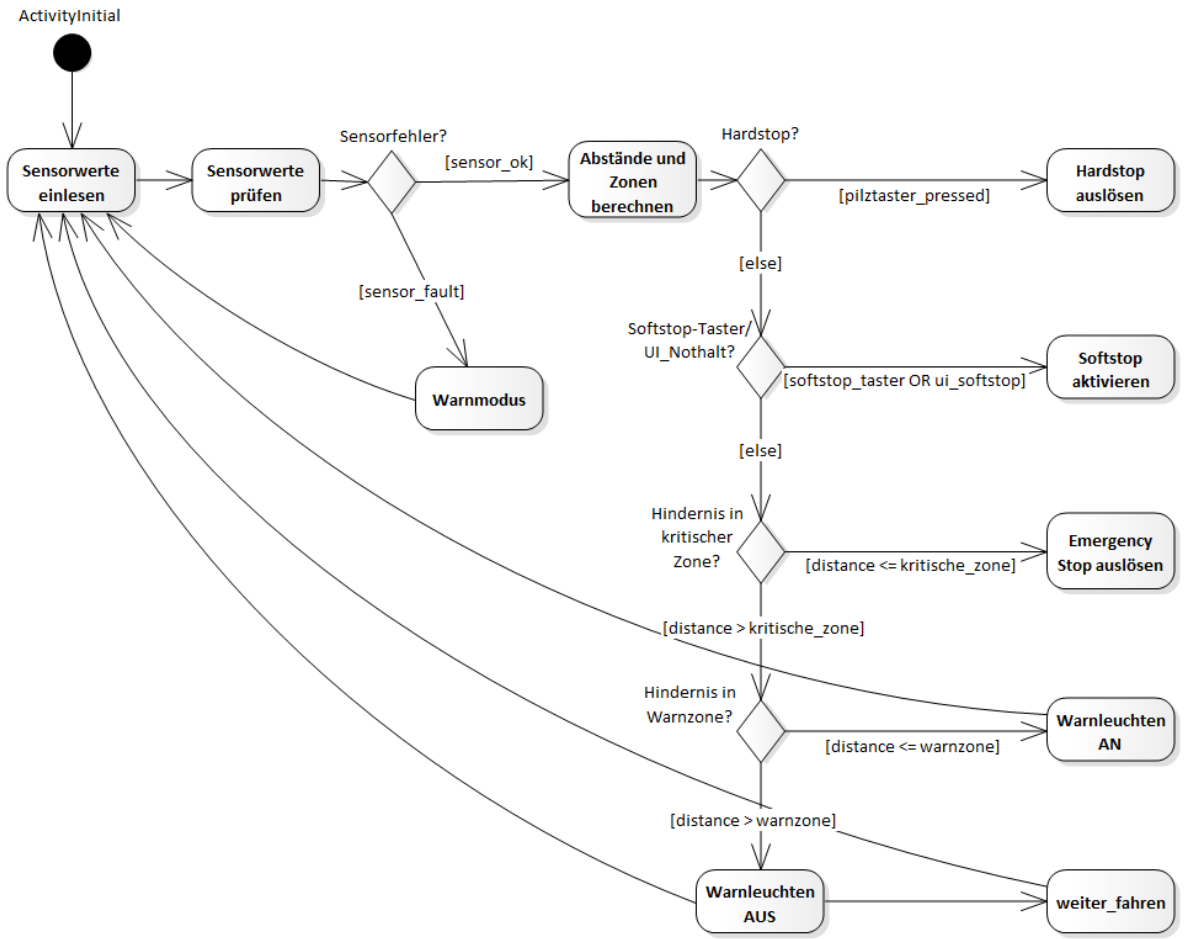

Figure 5: Aktivitätsdiagramm „Nothalt auslösen“

Das Aktivitätsdiagramm beschreibt den vollständigen Entscheidungsablauf des Nothaltesystems von der Erfassung der Sensordaten bis zur Auslösung sicherheitsrelevanter Reaktionen. Es zeigt den kontinuierlichen zyklischen Prozess, in dem Sensordaten geprüft, interpretierte Zonen bewertet und manuelle Eingaben des Operators berücksichtigt werden. Der Ablauf reflektiert die Priorisierung sicherheitskritischer Ereignisse (Hardstop > Softstop > automatischer E-Stop > Warnung > Weiterfahren).

Initialer Zustand

Der Ablauf beginnt im ActivityInitial-Knoten und führt unmittelbar in die erste Aktivität:

1. Sensorwerte einlesen

Das System erfasst in regelmäßigen Intervallen die aktuellen Sensordaten:

- LiDAR-Daten (2D/3D)
- Encoder-Werte
- eventuell weitere Sensordaten wie Zustandsmeldungen

Alle Daten werden gesammelt und in den Verarbeitungspipeline überführt.

2. Sensorwerte prüfen

In einer Validierungsphase werden die eingelesenen Werte auf Plausibilität und Integrität geprüft.

Typische Prüfungen:

- Fehlmessungen
- Zeitstempelkonsistenz
- Kommunikationsfehler
- strukturelle Datenfehler (z. B. fehlerhafte PointCloud)

3. Entscheidung: Sensorfehler?

Der erste Entscheidungsknoten überprüft, ob ein sensor_fault vorliegt.

`[sensor_fault]` → Warnmodus aktivieren
Bei fehlerhafter Sensorik ist ein autonomer Weiterbetrieb nicht sicher möglich. Das System wechselt in einen abgesicherten Zustand (Warnmodus).

`[sensor_ok]` → Abstände und Zonen berechnen
Der normale Nothalt-Ablauf wird fortgesetzt.

4. Abstände und Zonen berechnen

Das System bestimmt aus den Sensordaten die aktuellen Abstände zu Objekten und ordnet diese den Gefahrenzonen zu:

- kritische Zone (Sofortstopp erforderlich)
- Warnzone (nur optische Warnung, aber kein Stopp)
- freie Zone (keine Maßnahmen notwendig)

5. Entscheidung: Hardstop?

Der nächste Entscheidungsknoten prüft, ob der Hardstop-Taster betätigt wurde:

`[pilztaster_pressed]` → Hardstop auslösen
Diese Aktion hat höchste Priorität. Sie führt unmittelbar zur Notabschaltung (Energie aus / Fahrzeugstop).

`[else]` → Weiter zur Softstop-Prüfung

6. Entscheidung: Softstop-Taster oder UI-Nothalt?

Hier werden manuelle Softstop-Signale ausgewertet:

- physischer Softstop-Taster
- Softstop-Befehl über UI / RViz

`[softstop_taster OR ui_softstop]` → Softstop aktivieren
Der Softstop reduziert die Geschwindigkeit oder hält das Fahrzeug kontrolliert an.

`[else]` → Weiter zur Hindernisbewertung

7. Entscheidung: Hindernis in kritischer Zone?

Wenn ein Objekt in der kritischen Zone erkannt wird:

`[distance <= kritische_zone]` → Emergency Stop auslösen
Dies ist der automatische E-Stop, ausgelöst durch die Sensorik.

`[distance > kritische_zone]` → Weiter zur Warnzonenprüfung

8. Entscheidung: Hindernis in Warnzone?

Wenn ein Hindernis nicht kritisch, aber sicherheitsrelevant ist:

`[distance <= warnzone]` → Warnleuchten AN
Das Fahrzeug fährt weiter, aber signalisiert eine Gefahrenlage.

`[distance > warnzone]` → Warnleuchten AUS → weiter_fahren

9. Weiterfahren

Wenn weder Hardstop, Softstop, E-Stop noch Warnsignale notwendig sind, bleibt das Fahrzeug im normalen Fahrbetrieb und der Zyklus beginnt erneut.

6.3 Sequenzdiagramme

6.3.1 Sensorfehler/Warnmodus

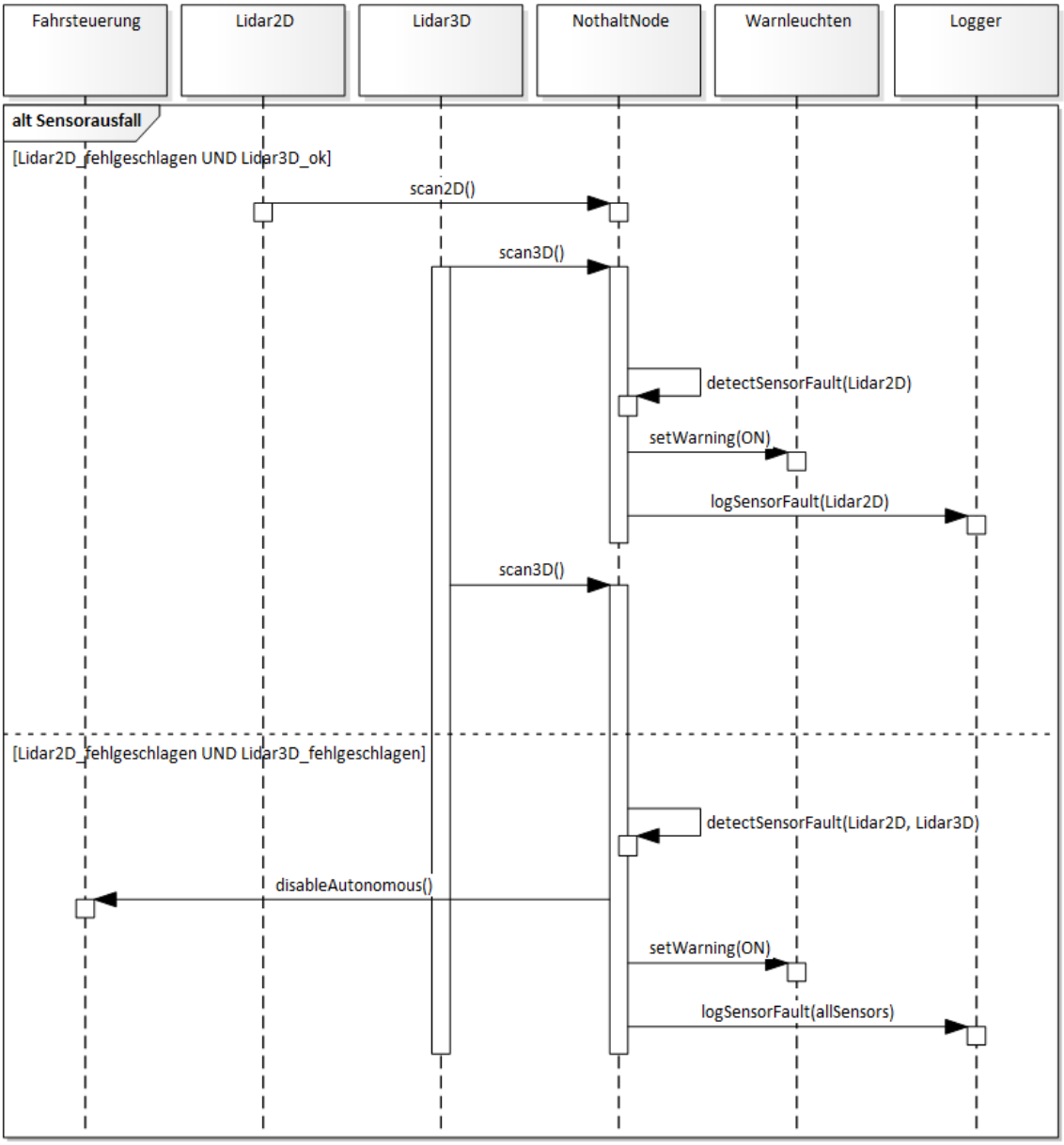

Figure 6: Sequenzdiagramm „Sensorfehler/Warnmodus“

Das Sequenzdiagramm beschreibt das Verhalten des Systems, wenn einer oder mehrere Lidarsensoren fehlschlagen. Die Verarbeitung unterscheidet dabei klar zwischen dem Aus-

fall eines einzelnen Sensors und dem gleichzeitigen Ausfall beider Sensorquellen.

Das Ziel dieses Modells ist es, den Übergang in den Warnmodus nachvollziehbar zu spezifizieren, ohne einen Hardstop auszulösen, solange zumindest eine valide Sensorquelle zur Verfügung steht.

Ablaufbeschreibung

1. Zyklische Sensordatenerfassung Sowohl der 2D-Lidar als auch der 3D-Lidar senden kontinuierlich Sensordaten an die Nothalt-Node:

- scan2D()
- scan3D()

Diese Sensordaten bilden die Grundlage für die weitere Verarbeitung in der Entscheidungslogik.

2. Alternativer Entscheidungsblock (alt): Sensorfehler Fall A – 2D-Lidar ausgefallen, 3D-Lidar funktionsfähig

`[Lidar2D_fehlgeschlagen UND Lidar3D_ok]`

Ablauf:

1. Der 3D-Lidar liefert weiterhin scan3D() Daten.
2. Die Nothalt-Node erkennt den Fehler:
    - detectSensorFault(Lidar2D)
3. Das System aktiviert die Warnsignale:
    - setWarning(ON)
4. Der Logger dokumentiert den Fehler:
    - logSensorFault(Lidar2D)

Interpretation: Der autonome Betrieb darf vorsichtig weiterlaufen, da der 3D-Sensor als redundante Sicherheitsquelle weiterhin korrekt arbeitet.

Fall B – Beide Sensoren fehlerhaft

`[Lidar2D_fehlgeschlagen UND Lidar3D_fehlgeschlagen]`

Ablauf:

1. Die Nothalt-Node erkennt den vollständigen Ausfall:
   - detectSensorFault(Lidar2D, Lidar3D)
2. Aktivieren der Warnsignale:
   - setWarning(ON)
3. Protokollierung des kombinierten Fehlers:
   - logSensorFault(allSensors)
4. Die Fahrsteuerung wird informiert:
   - disableAutonomous()

Interpretation: Das Fahrzeug darf nicht mehr autonom bewegt werden. Im Zustandsdiagramm führt dies zum Blink-Signal mit Signalleuchten und keine Freigabe für Motorbewegung (Warnmodus).

3. Bedeutung im Gesamtsystem Der Ablauf stellt sicher, dass:

- nicht sofort Hardstop ausgelöst wird, wenn nur ein Sensor betroffen ist
- Redundanz genutzt wird, solange mindestens ein Sensor valide Daten liefert
- Warnmodus aktiviert wird, sobald keinerlei sichere Messdaten mehr vorliegen
- stets ein definierter sicherer Zustand erreicht wird
- Fehler systematisch dokumentiert werden

Zusammenfassung Das Sequenzdiagramm demonstriert:

1. die redundante Sensorverarbeitung,
2. die klare Unterscheidung zwischen teilweisem und vollständigem Sensorfehler,
3. die Aktivierung des Warnmodus statt eines sofortigen Hardstop,
4. die Weitergabe der Statusinformationen an Warnleuchten und Logger,
5. die sichere Deaktivierung des autonomen Fahrbetriebs bei Totalausfall.

6.3.2 Softstop-Sequenz

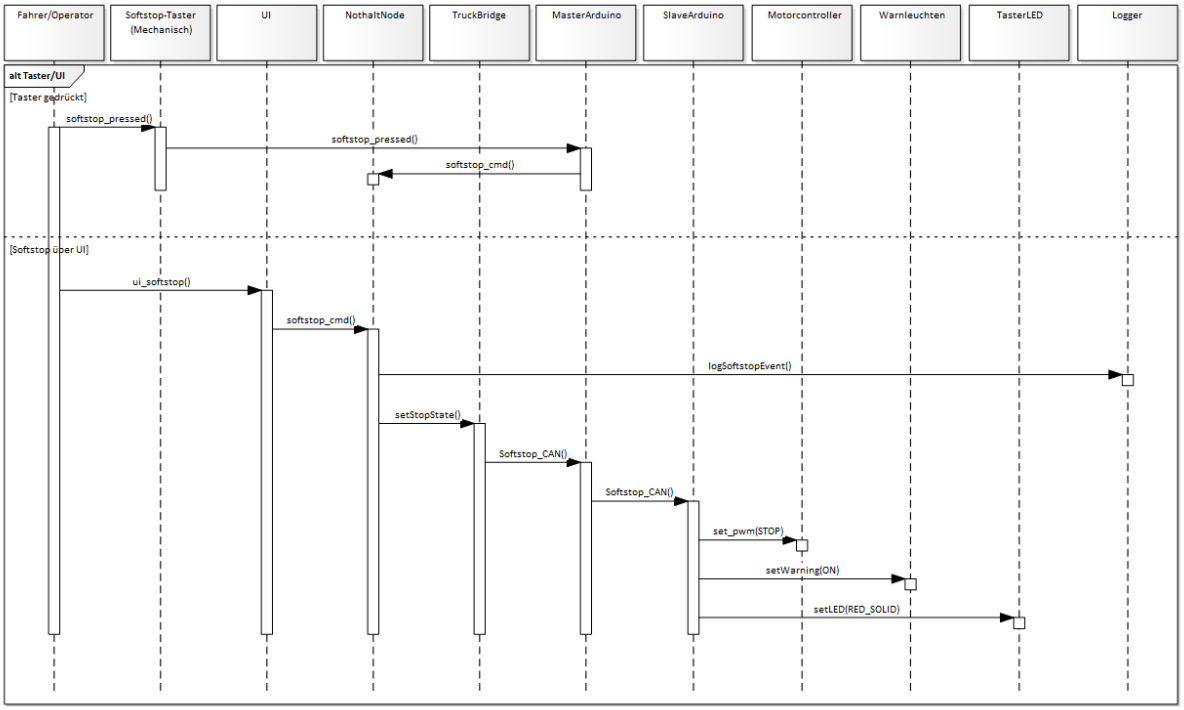

Figure 7: Sequenzdiagramm „Softstop"

Das Sequenzdiagramm zeigt den Ablauf eines Softstops, der entweder durch den mechanischen Softstop-Taster oder über die Benutzeroberfläche (UI) ausgelöst wird. Der Softstop stellt einen kontrollierten, nicht systemverriegelnden Stopp dar und hat geringere Priorität als Hardstop und E-Stop. Dennoch führt er zu einer sofortigen Reduktion

der Geschwindigkeit bzw. einem Anhalten des Fahrzeugs. Die Darstellung nutzt einen alt-Block zur Unterscheidung beider Auslösewege.

Ablaufbeschreibung

1. Alternativer Auslöser: Taster oder UI Fall A – Softstop über mechanischen Taster

`[Taster gedrückt]`

Ablauf:

1. Der Fahrer betätigt den mechanischen Softstop-Taster:
   - softstop_pressed()
2. Das UI zeigt die Aktivierung an (sofern gekoppelt).
3. Die Nothalt-Node empfängt das Tastensignal:
   - softstop_cmd()
4. Die interne State-Machine setzt Softstop als aktiven Zustand.

Fall B – Softstop über UI

`[Softstop über UI]`

Ablauf:

1. Softstop wird über das UI ausgelöst:
   - ui_softstop()
2. UI sendet den Befehl an die Nothalt-Node:
   - softstop_cmd()
3. Die Nothalt-Node setzt den Softstop-Zustand:
   - setStopState()
4. Weitergabe über CAN:
   - Softstop_CAN()

5. Weitergabe an MasterArduino:
   - Softstop_CAN()
6. Sicherungsaktionen:
   - set_pwm(STOP)
   - setWarning(ON)
   - setLED(RED_SOLID)
7. Logging:
   - logSoftstopEvent()

2. Wirkungskette des Softstops Der Ablauf macht klar:

- Softstop beeinflusst direkt die Aktorik (PWM STOP).
- Softstop aktiviert Warnleuchten, aber nicht den Warnmodus.
- Softstop aktiviert die Taster-LED, aber nicht die Verriegelung (wie beim Hardstop).
- Softstop wird vollständig über ROS → CAN → Arduino → Motorcontroller propagiert.

3. Protokollierung Das Logging sorgt für:

- Nachvollziehbarkeit bei Tests
- Diagnose von Fehlbedienungen
- Analyse in der Sicherheitstechnik

6.3.3 E-Stop-Sequenz

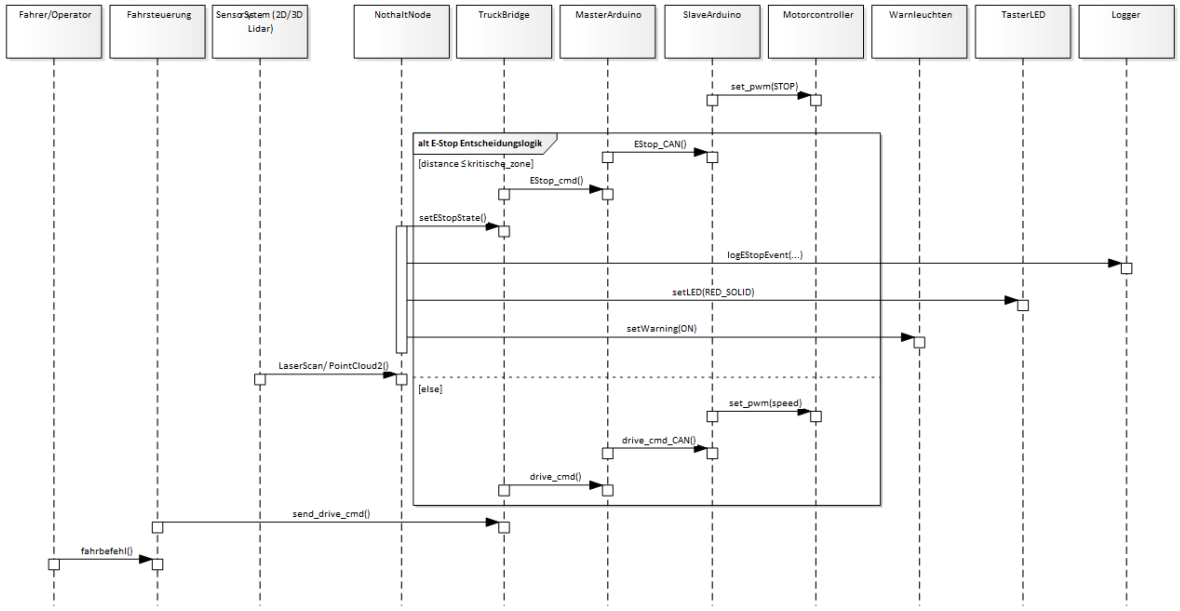

Figure 8: Sequenzdiagramm „E-Stop“

Ablaufbeschreibung

1. Sensorerfassung der Umgebung Das Sensorsystem übermittelt kontinuierlich Umgebungsdaten an die Nothalt-Node:

- LaserScan()
- PointCloud2()

2. E-Stop Entscheidungslogik Die Nothalt-Node verarbeitet die Sensordaten in einem alt-Block:

Fall A – Hindernis in kritischer Zone

Bedingung:

```
[distance <= kritische_zone]
```

Ablauf:

1. setEStopState()
2. EStop_cmd() an TruckBridge
3. EStop_CAN() an MasterArduino
4. MasterArduino:
   - setLED(RED_SOLID)
   - setWarning(ON)
   - logEStopEvent(...)
5. set_pwm(STOP) an SlaveArduino
6. Motorcontroller stoppt sofort

Fall B – Kein Hindernis in kritischer Zone

Bedingung:

```
[distance > kritische_zone]
```

Ablauf:

1. drive_cmd()
2. drive_cmd_CAN()
3. set_pwm(speed)

3. Fahrbefehle des Operators Fahrzyklus:

- fahrbefehl() vom Operator
- send_drive_cmd() an Nothalt-Node

4. **Logging des E-Stop-Ereignisses** Der Logger speichert:
   - Zeitpunkt
   - Sensorwerte
   - Auslöser
   - Fahrzeugstatus
   - Aktorreaktionen

Zusammenfassung Das Sequenzdiagramm zeigt:

1. automatischer E-Stop durch kritische Zone
2. vollständige Sicherheitskette (Master/Slave/Controller)
3. sofortiger Motorstopp
4. Aktivierung der Warnsignale
5. Protokollierung

6.3.4 Quittierung & Kriechmodus

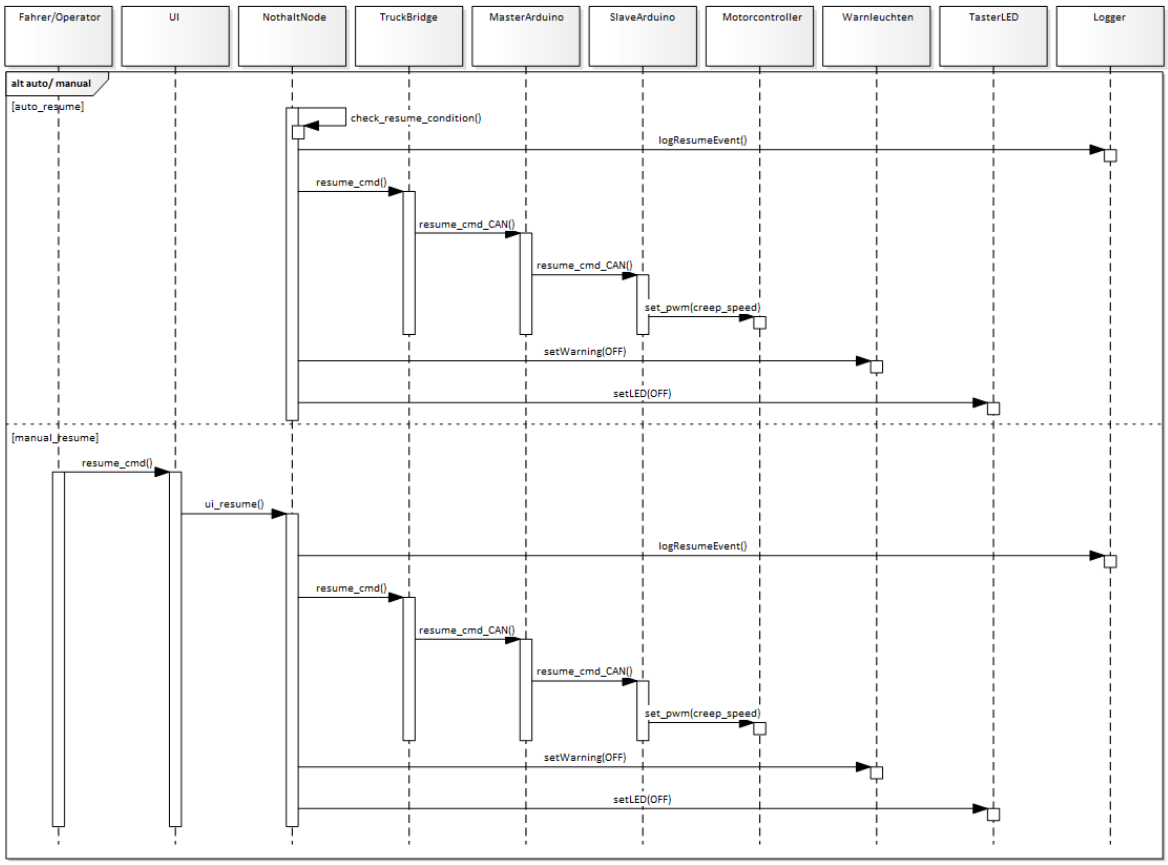

Figure 9: Sequenzdiagramm „Quittierung & Kriechmodus“

Das Sequenzdiagramm zeigt den Ablauf, mit dem das Fahrzeug den Fahrbetrieb nach einem Softstop oder E-Stop wieder aufnehmen kann. Die Wiederaufnahme kann automatisch erfolgen, wenn alle Sicherheitsbedingungen erfüllt sind, oder manuell durch den Operator.

Das Diagramm unterscheidet:

- Auto-Resume
- Manual-Resume

Fall A – Auto-Resume Bedingung:

`[auto_resume]`

Ablauf:

1. check_resume_condition()
2. resume_cmd()
3. resume_cmd_CAN()
4. MasterArduino:
   - set_pwm(creep_speed)
   - setWarning(OFF)
   - setLED(OFF)
5. logResumeEvent()

Fall B – Manual-Resume Bedingung:

`[manual_resume]`

Ablauf:

1. resume_cmd() über UI
2. resume_cmd_CAN()
3. MasterArduino:
   - set_pwm(creep_speed)
   - setWarning(OFF)
   - setLED(OFF)
4. logResumeEvent()

Kriechmodus Beide Wege führen zu:

- set_pwm(creep_speed)

Damit entsteht ein sicherer, langsamer Start:

- minimale Geschwindigkeit
- volle Operator-Kontrolle
- geeignet für enges Rangieren

Systemwirkungen und Konsistenz Das Sequenzdiagramm ist konsistent mit:

- Zustandsdiagrammen (Softstop/E-Stop → Resume → Kriechmodus)
- Aktivitätsdiagrammen
- Requirements (FR-SOFTSTOP, FR-ESTOP, FR-Kriechmodus, FR-INT-CAN)
- Sicherheitspriorisierung

Zusammenfassung Dieses Sequenzdiagramm bildet alle relevanten Aspekte des Wiederanfahrens ab:

1. automatische Wiederaufnahme
2. manuelle Wiederaufnahme
3. vollständige Weitergabe über ROS → CAN → Arduino
4. sichere Freigabe in den Kriechmodus
5. Abschalten der Warnsignale
6. Protokollierung aller Resume-Ereignisse

7 Systemarchitektur

7.1 Komponentenübersicht

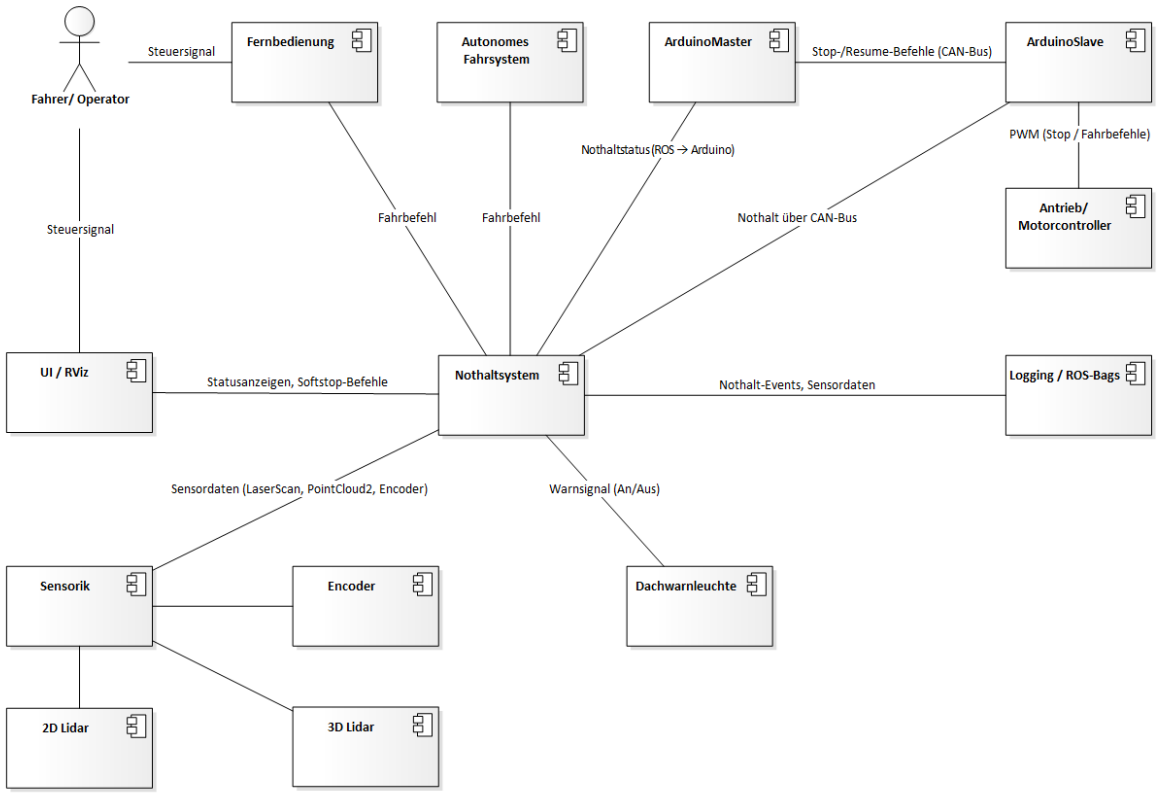

Figure 10: Komponentenübersicht des Nothaltesystems

Die Architektur des Nothaltesystems basiert auf einer klar strukturierten Aufteilung in Sensorik, zentrale Entscheidungslogik, Bedienkomponenten, Kommunikationsknoten sowie Aktorik. Das Klassendiagramm veranschaulicht die wesentlichen Systemkomponenten und ihre Beziehungen.

- **SensorSystem**
  Liefert die Distanzdaten der 2D- und 3D-LiDAR-Sensorik. Diese Informationen werden von der zentralen NothaltNode ausgewertet.

- **OperatorUI**
  Ermöglicht dem Operator das Auslösen von Softstop- und Resume-Befehlen über Taster oder grafische Benutzeroberfläche.

- **NothaltNode**
  Zentrale Komponente des Systems. Sie verarbeitet Sensorwerte, bewertet Gefahrenlagen, setzt Zustände (Hardstop/Softstop/E-Stop) und triggert sicherheitsrelevante Aktionen.

- **TruckBridgeNode**
  Vermittelt zwischen ROS und dem CAN-basierten Mikrocontrollersystem. Sie überträgt Nothalt- und Freigabekommandos zuverlässig an die Aktorik.

- **MasterArduino**
  Empfängt CAN-Kommandos und setzt sicherheitskritische Signale für den SlaveArduino.

- **SlaveArduino**
  Wandelt Sicherheitsbefehle in Motor-PWM-Signale um und steuert damit den Motorcontroller.

- **MotorController**
  Führt die tatsächliche Bremsung bzw. den Stopp aus.

- **Logger**
  Dokumentiert Nothalt-, Sensor- und Zustandsereignisse zur späteren Analyse.

Diese Komponenten bilden gemeinsam die technische Basis für einen deterministischen, nachvollziehbaren und sicherheitsorientierten Nothaltprozess.

7.2 Daten- & Funktionsflüsse

Das System folgt einer klaren Datenflusskette vom Erfassen der Umgebung bis zur Umsetzung des Nothalts:

1. **SensorSystem → NothaltNode**
   Die NothaltNode erhält kontinuierliche Distanzdaten und bewertet, ob sich Hindernisse in der Warn- oder kritischen Zone befinden.

2. **OperatorUI → NothaltNode**
   Manuelle Eingaben wie Softstop oder Resume werden direkt an die Entscheidungslogik übergeben.

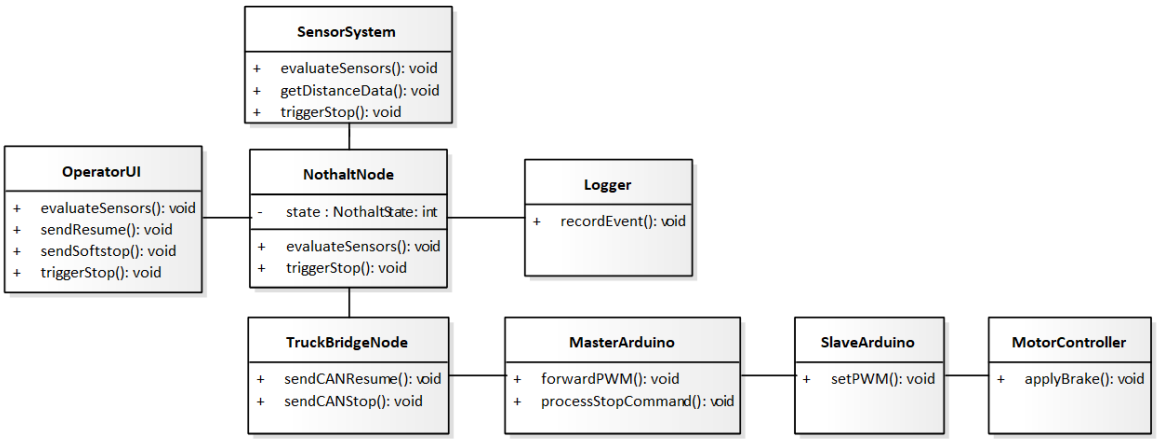

Figure 11: Systemarchitektur des Nothaltesystems

3. **NothaltNode → TruckBridgeNode**
   Wird ein Nothalt ausgelöst, sendet die NothaltNode entsprechende Kommandos an die CAN-Bridge.

4. **TruckBridgeNode → MasterArduino → SlaveArduino → MotorController**
   Der Befehl wird hardwareseitig weitergereicht, bis der Motorcontroller den Stopp ausführt.

5. **NothaltNode → Logger**
   Alle sicherheitsrelevanten Ereignisse (Softstop, Hardstop, E-Stop, Resume) werden aufgezeichnet.

Diese Struktur stellt sicher, dass die Nothaltlogik zentral ausgeführt wird, während die sicherheitskritische Aktorik über dedizierte Hardwarepfade zuverlässig angesprochen wird. Das Zusammenspiel der Klassen gewährleistet ein konsistentes, robustes und erweiterbares Systemdesign.

8 Qualitätssicherung der Anforderungen

8.1 Kriterien: Vollständigkeit, Eindeutigkeit, Konsistenz

Alle funktionalen und nicht-funktionalen Anforderungen wurden anhand folgender Kriterien geprüft:

- **Vollständigkeit**
  Es wurde sichergestellt, dass sämtliche sicherheitsrelevanten Funktionen (Hardstop, Softstop, E-Stop, Quittierung, Resume, Kriechmodus, Warnsignale, Logging) vollständig beschrieben und durch passende Anforderungen abgedeckt sind.

- **Eindeutigkeit**
  Jede Anforderung wurde so formuliert, dass nur eine mögliche Interpretation zulässig ist. Unklare Begriffe wie „schnell“, „zuverlässig“, „rechtzeitig“ wurden vermieden oder messbar definiert.

- **Nachvollziehbarkeit**
  Anforderungen wurden so strukturiert, dass sie eindeutig auf Use-Cases, UML-Diagramme und Systemfunktionen zurückgeführt werden können. Jede wesentliche Systemfunktion findet sich in mindestens einem Modell wieder.

- **Konsistenz**
  Die Anforderungen wurden auf Widersprüche geprüft. Prioritätsregeln (Hardstop > E-Stop > Softstop) wurden über alle Modelle hinweg einheitlich formuliert. Mehrfach vorkommende Anforderungen wurden zusammengeführt oder harmonisiert.

- **Verifizierbarkeit**
  Alle Anforderungen wurden so definiert, dass sie durch Tests, Simulationen oder statische Analyse überprüfbar sind. Insbesondere sicherheitsrelevante Anforderungen sind mit messbaren Bedingungen versehen.

8.2 Vorgehen zur Qualitätssicherung

Zur Sicherstellung der Qualität wurden folgende Schritte durchgeführt:

- **Iterative Review-Zyklen**
  Anforderungen, Diagramme und Systembeschreibungen wurden mehrfach überarbeitet. Jede neue Modellierung (Use Case, Sequenz-, Zustands- oder Aktivitätsdiagramm) diente als Spiegel zur Überprüfung der Anforderungen.

- **Abgleich mit Use-Case-Modell**
  Für jeden Use-Case wurde geprüft, ob die entsprechenden funktionalen Anforderungen vollständig vorhanden sind. Beispiele:

  – „Hardstop“ deckt FR-Hardstop ab

  - „Softstop“ deckt FR-Softstop und FR-Resume ab
  - „Hindernis erkennen“ deckt FR-E-Stop ab

- **Abgleich mit Zustandsdiagrammen**
  Die Zustandsmodelle dienten insbesondere dazu, die Vollständigkeit der Systemreaktionen zu prüfen (z. B. Verhalten im Hardstop, Übergänge im Softstop und E-Stop, Wiederaufnahmebedingungen).

- **Abgleich mit Sequenzdiagrammen**
  Die Sequenzdiagramme zeigen die tatsächlichen Nachrichtenflüsse und Systeminteraktionen. Dadurch konnten unvollständige oder unpräzise Anforderungen identifiziert und angepasst werden.

- **Validierung durch Architekturabgleich**
  Die Anforderungen wurden mit der Systemarchitektur (ROS-Nodes, CAN-Brücke, Arduino-Mikrocontroller, Motorcontroller, Sensorik) abgeglichen. Ziel war: jede Anforderung muss technisch realisierbar und logisch zum Systemaufbau passend sein.

8.3 Verifikation der Anforderungen

Da im Rahmen dieser Projektarbeit ausschließlich eine Software Requirements Specification erstellt wurde, erfolgt die Verifikation der Anforderungen rein dokumentarisch und modellbasiert. Es wurden keine praktischen Tests oder Implementierungen durchgeführt.

Die Verifikation basiert auf drei Maßnahmen:

1. **Modellbasierte Prüfung**
   Die UML-Diagramme (Use Case, Aktivität, Zustand, Sequenz) wurden genutzt, um die Anforderungen auf logische Konsistenz, Vollständigkeit und korrektes Systemverhalten zu überprüfen.

2. **Dokumentarische Konsistenzprüfung**
   Die Anforderungen wurden iterativ überarbeitet und auf Eindeutigkeit, Widerspruchsfreiheit und klare Zuordnung zu Systemfunktionen geprüft.

3. **Abgleich mit der Systemarchitektur**
   Die Anforderungen wurden mit der modellierten Architektur verglichen, um sicherzustellen, dass alle Funktionen technisch sinnvoll abbildbar sind und alle Schnittstellen konsistent beschrieben werden.

Diese Maßnahmen gewährleisten, dass die spezifizierten Anforderungen nachvollziehbar, konsistent und verifizierbar formuliert sind.

8.4 Nachvollziehbarkeit (Traceability)

Zur Sicherstellung der Nachvollziehbarkeit besteht eine direkte Beziehung zwischen:

- funktionalen Anforderungen (FR-xx)
- Use Cases
- Zustandsdiagrammen
- Sequenzdiagrammen
- Architekturkomponenten

Jede sicherheitsrelevante Anforderung taucht in mindestens einem UML-Diagramm wieder auf.

Beispiele:

- FR-HS-xx → Use Case „Hardstop betätigen“, Sequenzdiagramm Hardstop, Zustandsdiagramm
- FR-SS-xx → Use Case „Softstop betätigen“, Softstop-Sequenzdiagramm, Zustandsdiagramm
- FR-ES-xx → Use Case „Hindernis erkennen (E-Stop)“, E-Stop-Sequenzdiagramm, Aktivitätsdiagramm
- FR-RM-xx → Substatemaschinen Softstop/E-Stop + Resume-Sequenzdiagramm

Damit ist gewährleistet, dass jede Anforderung eindeutig auf modellierte Systemfunktionen rückführbar ist.

Definitionen

Hardstop Physicher Not-Aus über Pilztaster; verriegelt das System und setzt Motorcontroller sofort außer Betrieb

Softstop Kontrollierter Stopp des Fahrzeugs durch Taster oder UI; keine Systemverriegelung

E-Stop Automatischer Notstopp durch Sensorik bei kritischem Hindernisabstand

Resume Wiederaufnahme des Fahrbetriebs durch automatische oder manuelle Freigabe

Kriechmodus Langsame, begrenzte Bewegung nach einem Nothalt zur sicheren Situationsklärung.

LED Light Emitting Diode

UI User Interface

PWM Pulse Width Modulation; Pulsweitenmodulation zur Steuerung von Motoren und Aktoren

ROS Robot Operating System; flexibles Framework für die Entwicklung von Robotersoftware

CAN Controller Area Network; serielles Bussystem zur Kommunikation zwischen Mikrocontrollern

LiDAR Light Detection and Ranging; optisches Messverfahren zur Entfernungsermittlung mittels Laserlicht

UML Unified Modeling Language

Warnmodus Blink-Signal mit Signalleuchten und keine Freigabe für Motorbewegung